AN ESSAY FOR BUILDERS

Why We Need to Stop Dropping Every Prompt onto GPUs

Most enterprise prompts are lookups. The expensive mistake is sending all of them through a generative model.

Amin Parva · 20 October 2026 · cpu-llm 1.9.0



FIGURE 1 *A cluster kept hot for every prompt has a physical appetite: electricity, cooling, and a machine that stays on for questions a lookup could have answered.*

In 2024 the International Energy Agency compared a typical Google search, about 0.3 watt-hours, with a ChatGPT request, about 2.9 watt-hours. That comparison is the source of the familiar “ten times” line. Later measurements put a short, modern query much closer to a search, and a long or reasoning-heavy query well above it. The waste is not the hard question. The waste is sending a date lookup, an invoice check, or a record filter through that same machine.

Software is in an AI fever. The default design for a feature has become a multi-billion-parameter model and a GPU cluster.

- Need a date in a PDF? Call a frontier model.
- Need to know whether an invoice total matches? Send it to a GPU.
- Need to filter records the database already holds? Prompt a 70-billion-parameter model.

That default raises power and cloud cost, and it adds a wait that a lookup never needed. Most of those prompts do not need a generative model.

The physical cost

Treat the published hardware numbers as nameplate limits, not as a measurement of every query.

- **Power.** An NVIDIA DGX H100 system is specified at about 10.2 kW maximum. That is an eight-GPU server: each H100 SXM is a 700 W part, and the rest of the node is CPUs, memory, networking, and cooling. At that ceiling it is in the range of several home air conditioners left on together. Real workloads often sit below the nameplate. The machine is still in that class of power while it waits.
- **Water.** Evaporative cooling is the second bill. NVIDIA has put conventional cooling-tower use near 2.6 million gallons per megawatt per year, and has described newer dry-cooler designs that cut evaporative water to nearly nothing. The design question is why a date-in-a-PDF query is on a machine that needs either system.
- **Latency.** A hosted generative call often spends hundreds of milliseconds, and sometimes seconds, before the first token. On a warmed cpu-llm process, a small diagnostic run measured a 37.6 ms median and a 58.9 ms p95, on an Intel Core Ultra 9 275HX, with encoding included and with startup excluded. Cold start is a different number: loading the encoder took 11.5 seconds in an earlier run. Load it once and reuse the process.

cpu-llm is not a small LLM

cpu-llm (source) is an embeddable Python library for question answering over documents you already hold. Version 1.9.0 runs on Python 3.11 or newer and a 64-bit CPU, with PostgreSQL and pgvector. It does not call OpenAI, Gemini, or Anthropic, and it is built to run with no GPU.

Despite the name, it does not sample the next token. A frozen all-mpnet-base-v2 encoder runs on CPU. Each paragraph is stored as a 1024-dimensional VectorPrism vector. Search uses the dense slice of that vector, then re-ranks. For question forms the library recognizes — definitions, attributes, locations, weights, and simple event questions such as color, place, and who did what — the reply is a checked sentence, a source quotation, or an explicit non-answer. If the retrieved passages conflict, that conflict is the result.

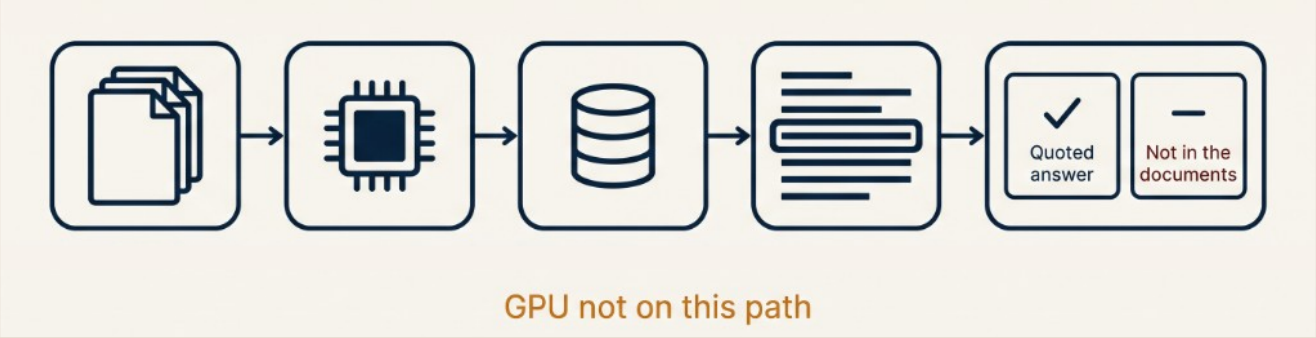


FIGURE 2 *Files are encoded on CPU, stored in PostgreSQL with pgvector, and answered from a stored sentence — or the library says the documents do not contain the answer. The GPU is not on this path.*

Scanned pages are out of scope: there is no OCR pipeline. The grammars are narrow English forms. Open chat, translation, and code generation are outside what the library demonstrates.

What that is good for

- **No GPU on the lookup path.** The process is CPU-only by construction, including a guard that will not start if CUDA is already visible.
- **A gap stays a gap.** Recognized questions abstain or surface a conflict instead of inventing a fact. That is the contract for those forms. It is not a universal proof that every string the process can emit is true. Evaluate it on your own documents. Creative mode can vary between runs, and a local FLAN-T5 checkpoint, if one is installed, can be used on the rule path.
- **A quote you can open.** Paragraphs live in your database, under a tenant namespace you choose. A local authenticated HTTP entry exists for callers that are not already trusted code.
- **A few jobs that are not chat.** Supported arithmetic is evaluated as arithmetic. Policy-style questions can return a bounded verdict such as YES, NO, UNKNOWN, or CONTESTED. Supplied-field finance checks run on the fields you pass in.

Accepted files are plain text, Markdown, HTML, PDF, DOCX, CSV, XLSX, XLSM, and EPUB. Readers extract text that is already in the file.

The migration that actually happens

Teams tend to walk the same three steps.

- **Public model APIs.** Fast to try. At volume, the bill tracks tokens, and compliance teams start asking where the documents went.
- **Self-hosted GPUs.** Open models on your own cluster, or on a rented GPU node, keep the data closer and can lower the per-token price. The new cost is a 700 W-class accelerator, often an entire 10 kW-class server, held ready for prompts that never needed it.
- **A router in front of the model.** A fast CPU classifier decides what the prompt actually is. Lookups, exact checks, and recognized document questions stay on CPU. Compact models handle light parsing. The GPU cluster is reserved for work that needs open-ended reasoning.

Call that a hyper-agentic layout if you want a name: agents where an agent is warranted, and deterministic automation everywhere else. Inside cpu-llm, the same idea shows up as a decision to answer, ask for a missing fact, refuse, or escalate, before anything is shown to the user.

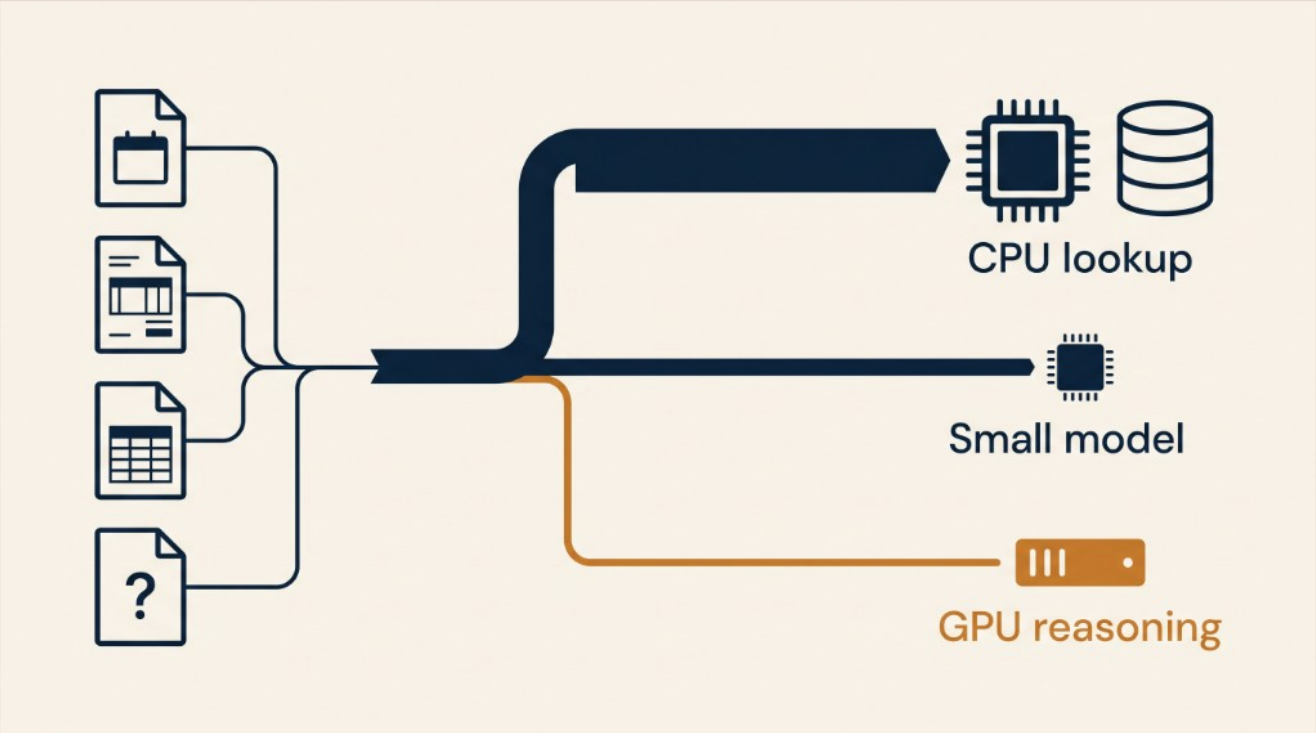


FIGURE 3 *One front door, three lanes. Read it left to right. Routine documents stay on the CPU lookup. A thinner lane uses a small model. The GPU is the remaining lane, drawn in amber.*

A planning split many teams can start from, and then measure on their own traffic:

- **Routine lookups** go to SQL, pgvector, or cpu-llm. Warmed latency is on the order of tens of milliseconds. No GPU.
- **Light parsing** goes to a small model on CPU or an NPU.
- **Deep reasoning** is the traffic that should wake the GPU cluster.

Do not adopt 80/15/5, or any other ratio, as an industry fact. Log a week of production prompts and set the split from that log.

Side by side

The cells below are either published hardware specifications or measurements from this project. They are not a price quote and not a service-level agreement.

Dimension	Generative path on a GPU	cpu-llm on CPU
What actually runs	A generative model, usually behind an API or on a rented GPU node.	A frozen CPU encoder, PostgreSQL with pgvector, and a checked sentence.
Nameplate power	A DGX H100-class server is specified at about 10.2 kW maximum. An H100 SXM is a 700 W part.	No GPU is attached to this path. The process refuses to start if CUDA is already visible.
When the documents are silent	A fluent sentence is still possible.	Recognized question forms quote a stored sentence, or they say the corpus does not settle it.
A measured lookup	Often hundreds of milliseconds to the first token, and sometimes seconds. Not one universal number.	37.6 ms median and 58.9 ms p95 on one warmed workstation run. Encoder startup was 11.5 s and is paid once.

What to take from this

- **Stop sending simple prompts through a generative model.** If the database, a vector index, or a rule can answer, that is the system of record.
- **cpu-llm is one demonstration that document questions can run on a CPU:** retrieve a paragraph, rebuild only what that paragraph supports, or say the corpus does not settle it.
- **Efficiency is a routing problem.** A larger GPU pool does not fix a front door that treats every prompt as a reasoning task.

Pair a deterministic CPU path with a generative model and the model gets a smaller, more defensible job. The GPU is still there. It is no longer the default.

Sources

International Energy Agency, Electricity 2024. The widely cited comparison of about 0.3 Wh for a search and about 2.9 Wh for a ChatGPT request.

Epoch AI, How much energy does ChatGPT use? A later estimate of about 0.3 Wh for a typical short GPT-4o query, and a much higher figure for long inputs.

NVIDIA, DGX H100 datasheet. System power about 10.2 kW maximum.

NVIDIA, liquid cooling for AI factories. About 2.6 million gallons per megawatt per year for cooling-tower designs, and dry-cooler designs aimed at near-zero evaporative use.

cpu-llm latency and scope: warmed in-process median 37.6 ms and p95 58.9 ms at commit 1183b77, Intel Core Ultra 9 275HX; encoder startup of 11.5 s from the 18 October 2026 question-time note. Package and repository: pypi.org/project/cpu-llm and github.com/insightitsGit/cpullm. These figures describe those runs. They are not an API service level. Apache-2.0.

The 80/15/5 split sometimes quoted for this kind of router is a planning sketch. It is not a measurement from cpu-llm or from a cited study.